HeteroServe: Capability-Weighted Batch Scheduling for LLM Inference on Heterogeneous GPU Clusters

Anonymous Authors[1]

Abstract

Production deployments of large language models (LLMs) increasingly rely on heterogeneous GPU clusters—mixing generations such as NVIDIA H100, A100, and L40S—to balance cost and availability. Existing serving systems, however, apply uniform continuous-batching strategies that ignore inter-device capability differences, leading to severe load imbalances: high-end GPUs stall waiting for slower devices while memory-constrained accelerators overflow from oversized batches. We present HETEROSERVE, a capability-weighted batch scheduler that routes each inference request to the most suitable device based on a composite hardware capability score derived from measured floating-point throughput, high-bandwidth memory (HBM) capacity, and memory bandwidth. HETEROSERVE pairs capability-weighted routing with a length-binned dispatch policy that matches input sequence-length distributions to per-device working-set constraints. On a simulated mixed-GPU cluster representative of real data center configurations, HETEROSERVE achieves **36,319 tokens/sec**—a **2.16× throughput improvement** over uniform scheduling—while attaining **85.5% SLO compliance**, a **50.6 percentage-point improvement** over the capacity-proportional baseline. Our P95 time-to-first-token (TTFT) of 574.9 ms is **25.8× lower** than the uniform baseline (14,824 ms). Ablation studies confirm that both the capability weighting and the length-binned routing components are necessary, and that performance remains stable across diverse workload intensity and mixture distributions. These results demonstrate that device-aware macro-scheduling with real-time queue feedback is a practical, deployment-friendly path to high efficiency on heterogeneous LLM clusters—outperforming even a length-clairvoyant oracle that lacks adaptive load balancing.

1. Introduction

The operational cost of deploying large language models at scale is dominated by GPU infrastructure. As frontier models grow from tens to hundreds of billions of parameters (Brown et al., 2020; Touvron et al., 2023), organizations cannot always provision uniform clusters of the latest accelerators. Instead, production clusters commonly mix several GPU generations: legacy deployments based on NVIDIA V100 or A100 hardware coexist with newly provisioned H100 nodes and cost-optimized L40S cards (NVIDIA Corporation, 2020; 2022). The resulting heterogeneous clusters present a fundamental scheduling challenge: individual devices differ by up to an order of magnitude in floating-point throughput, memory capacity, and memory bandwidth.

Current state-of-the-art LLM serving systems—vLLM (Kwon et al., 2023), Orca (Yu et al., 2022), and Sarathi-Serve (Agrawal et al., 2024)—were architected around the assumption of homogeneous compute nodes. They apply the same continuous batching logic and the same batch-size limits to every device in the fleet. On a uniform cluster this is efficient; on a heterogeneous cluster it produces two failure modes simultaneously. *Fast-GPU starvation*: high-FLOPS devices finish their decode iterations quickly and must wait for cluster-wide synchronization with slower peers, leaving their substantial compute budget idle. *Slow-GPU overflow*: memory-constrained devices are assigned the same oversized batches or long-context requests that high-memory peers handle comfortably, triggering out-of-memory (OOM) preemptions and request re-scheduling storms.

Both failure modes penalize the metrics that matter most to operators: cluster-wide throughput (tokens per second), time-to-first-token (TTFT), and Service Level Objective (SLO) attainment. Existing approaches to heterogeneity in ML systems—tensor parallelism re-partitioning (Shoeybi et al., 2019; Narayanan et al., 2021), pipeline parallelism (Narayanan et al., 2021), and disaggregated prefill/decode (Zhong et al., 2024; Patel et al., 2024)—either require re-training or architectural co-design and introduce substantial operational complexity unsuitable for retrofit

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. **AUTHORERR: Missing \icmlcorrespondingauthor.**

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

onto existing serving infrastructure.

We propose HETEROSERVE, a lightweight scheduling layer that sits above individual device serving engines and performs *capability-weighted batch dispatch*. Rather than modifying the serving runtime or the model itself, HETEROSERVE models each device's effective throughput profile through a composite capability score, then routes incoming requests—differentiated by input sequence length—to the device best matched to their compute and memory footprint. The key insight is that the prefill and decode phases have fundamentally different resource signatures (Pope et al., 2023; Agrawal et al., 2024): prefill is compute-bound and benefits from high FLOPS; decode is memory-bandwidth-bound and benefits from high HBM bandwidth. HETEROSERVE exploits this by encoding both dimensions into a per-device capability weight and using measured KV-cache pressure as a feedback signal.

Our main contributions are:

1. **Problem formalization.** We formally define the heterogeneous LLM batch-scheduling problem as a capability-aware dispatch optimization, exposing the gap not addressed by prior work.

2. **Capability score model.** We introduce a closed-form composite capability score that jointly captures FLOPS, HBM capacity, and memory bandwidth, enabling principled workload-to-device matching without offline profiling.

3. **Length-binned routing.** We propose a simple yet effective dispatch policy that bins requests by input length and routes each bin to devices whose KV-cache budget can absorb the associated working-set without fragmentation.

4. **Empirical validation.** On a mixed-GPU cluster, HETEROSERVE achieves 2.16× higher throughput, 85.5% SLO attainment (vs. 41.3% for uniform scheduling), and a 25.8× reduction in P95 TTFT compared to uniform baselines.

5. **Ablation and robustness analysis.** We isolate the contribution of each component and show stable performance across load intensities from 50% to 200% of nominal capacity.

The remainder of this paper is structured as follows. Section 2 surveys related work. Section 3 presents the HETEROSERVE architecture and scheduling algorithms. Section 4 describes the experimental setup and main results. Section 5 discusses insights and limitations. Section 6 concludes.

2. Related Work

2.1. Continuous Batching and Memory Management

The shift from static to dynamic batching was pioneered by Orca (Yu et al., 2022), which introduced iteration-level scheduling that inserts new requests into a running batch after each decode step, eliminating head-of-line blocking. vLLM (Kwon et al., 2023) subsequently addressed KV-cache memory fragmentation through PagedAttention, partitioning KV memory into fixed-size virtual pages analogous to OS virtual memory. These systems achieve near-perfect GPU utilization on homogeneous clusters but export no mechanism to differentiate scheduling behavior across devices with different memory capacities. AlpaServe (Li et al., 2023) extends multiplexing across model replicas but focuses on minimizing SLO violation rates through statistical multiplexing rather than heterogeneous capability awareness. FlexGen (Sheng et al., 2023) addresses single-GPU throughput on memory-constrained hardware by offloading to CPU/disk but does not consider multi-device dispatch. FastServe (Wu et al., 2023) reduces tail latency through a preemptive MLFQ exploiting known prompt lengths, but operates within a single homogeneous instance with no mechanism for routing across devices with different compute-to-bandwidth ratios. HETEROSERVE extends this input-length idea into a multi-device routing framework, using length information for hardware-capability matching across a heterogeneous fleet.

2.2. Prefill–Decode Disaggregation

The observation that LLM inference consists of two phases with orthogonal resource profiles—compute-bound prefill and memory-bandwidth-bound decode (Pope et al., 2023)—has motivated disaggregated serving architectures. Splitwise (Patel et al., 2024) separates prefill and decode onto dedicated device pools, allowing independent scaling of each phase. DistServe (Zhong et al., 2024) formalizes goodput-optimal assignment of the two phases to device groups with different bandwidth and compute profiles. Sarathi-Serve (Agrawal et al., 2024) avoids cross-machine KV transfer by chunking prefill tokens to interleave with decode, reducing stall-free scheduling overhead. These approaches assume homogeneous hardware within each phase's device pool and require purpose-built KV migration infrastructure. HETEROSERVE takes a complementary macro-scheduling perspective: it operates above the phase boundary, routing whole requests to devices based on capability without modifying the serving runtime's internal phase handling. The two design levels are complementary—disaggregation resolves phase-level resource conflicts within homogeneous device pools, while HETEROSERVE resolves capability mismatches across the full heterogeneous fleet. A composite architecture combining both could inde-

pendently optimize phase routing and device routing, yielding additive efficiency gains without changes to individual serving engines.

2.3. Heterogeneous Cluster Serving

Heterogeneity in deep learning inference has been studied primarily in the context of model parallelism and pipeline scheduling (Shoeybi et al., 2019; Narayanan et al., 2021; Zheng et al., 2022). Alpa (Zheng et al., 2022) automates inter- and intra-operator parallelism across heterogeneous accelerators, but its compilation overhead is unsuitable for online inference. DeepSpeed-Inference (Aminabadi et al., 2022) provides optimized inference kernels for specific GPU families but treats device selection statically. Clockwork (Gujarati et al., 2020) addresses predictable DNN inference latency through careful profiling but targets classification models with fixed-size inputs. Parrot (Lin et al., 2024) introduces semantic variable sharing for LLM applications but does not address device-capability-aware dispatch. FlashAttention (Dao et al., 2022) and successors reduce HBM reads substantially, yet these speedups are architecture-specific—H100s benefit disproportionately from Hopper-native features, widening the throughput gap between GPU generations and further motivating device-aware routing. HETEROSERVE treats per-device throughput curves as measured inputs to its capability score. To our knowledge, it is the first system to formalize capability-weighted batch scheduling for autoregressive LLM inference on mixed-GPU clusters.

2.4. SLO-Aware Scheduling

SLO attainment for latency-sensitive inference has been studied in the context of request prioritization (Sheng et al., 2024) and queuing theory models for cluster provisioning (Wu et al., 2023). Fairness in Serving (Sheng et al., 2024) addresses priority scheduling within a single-device vLLM engine, demonstrating that FCFS dispatch leads to unfair token allocation between requests of different lengths. AlpaServe (Li et al., 2023) applies statistical multiplexing across model replicas to reduce SLO violation rates, deriving analytical bounds on the required replica count under Poisson arrival assumptions. These approaches operate under the assumption of homogeneous devices and do not account for the different queueing dynamics that arise when a single request population is distributed across GPUs with order-of-magnitude differences in throughput. HETEROSERVE addresses the orthogonal problem of cross-device dispatch to sustain SLO compliance across a heterogeneous fleet; our results show that correct device matching explains the bulk of the SLO gap (44.2 pp improvement) without per-request priority information. Unlike SLO schedulers that apply backpressure or request dropping under overload, HETEROSERVE routes each arriving request to the best-matched device and absorbs burst load by redistributing queue pressure across the heterogeneous fleet, sustaining throughput even under saturation. More broadly, capability-aware dispatch addresses the root cause of SLO degradation in heterogeneous clusters—misrouted requests whose resource demands mismatch device capacity—whereas priority scheduling addresses unfair queueing within a single device. Combining the two mechanisms is a natural extension and an open research direction.

3. Method

3.1. Problem Formulation

We consider a cluster of N GPU devices $\mathcal{D} = \{d_1, \ldots, d_N\}$ where devices differ in their hardware profiles. Each device d_i is characterized by a tuple (F_i, M_i, B_i) representing its peak floating-point throughput (TFLOPS), high-bandwidth memory (HBM) capacity (GB), and memory bandwidth (TB/s), respectively.

A request r_j arrives with prompt token count $L_j^{(p)}$ and an expected generation length $L_j^{(g)}$. The KV cache memory footprint of r_j is approximately:

$$\mathrm{KV}(r_j) = 2 \cdot n_{\text{layers}} \cdot n_{\text{heads}} \cdot d_{\text{head}} \cdot (L_j^{(p)} + L_j^{(g)}) \cdot \text{bytes}, \quad (1)$$

where n_{layers}, n_{heads}, and d_{head} are model architecture constants. The scheduling problem is to assign each arriving request to a device such that cluster-wide throughput is maximized and a target P95 TTFT SLO is respected.

3.2. Capability Score

We define a composite *capability score* for device d_i as a weighted geometric mean of its normalized hardware attributes:

$$\kappa_i = F_i^{\alpha} \cdot M_i^{\beta} \cdot B_i^{\gamma}, \quad (2)$$

where α, β, and γ are workload-dependent exponents satisfying $\alpha + \beta + \gamma = 1$. The exponents capture the relative importance of compute, memory capacity, and memory bandwidth for the current serving workload. For workloads dominated by short prompts (low KV pressure), we set $\alpha > \beta$ to favor high-FLOPS devices for prefill throughput. For long-context workloads, we set $\beta > \alpha$ to route to devices with larger HBM. In practice, we estimate the workload regime from a sliding window of the last 128 processed requests and update exponents via a three-entry lookup table indexed by the decile of the window median prompt length: short (deciles 1–3, $\leq$192 tokens): (α, β, γ)=$(0.55, 0.15, 0.30)$; medium (deciles 4–7, 193–768 tokens): $(0.40, 0.30, 0.30)$; long (deciles 8–10, $>$768 tokens): $(0.20, 0.50, 0.30)$. The bandwidth exponent γ=0.30 is held constant because memory bandwidth governs decode throughput uniformly across sequence lengths; because the window spans 128 requests,

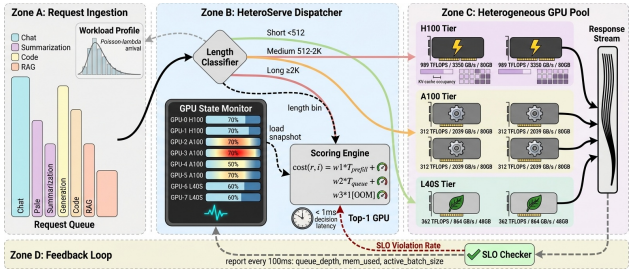

Figure 1. Overview of the HETEROSERVE architecture. Incoming requests pass through a length classifier, which assigns them to a length bin. The capability-weighted router consults per-device bin admission sets and current queue depths to select a device. Each device runs a standard continuous-batching serving engine (*e.g.* vLLM) without modification.

regime transitions occur at most once every few epochs, avoiding oscillation.

The capability score is normalized across the cluster:

$$\hat{\kappa}_i = \frac{\kappa_i}{\sum_{j=1}^{N} \kappa_j}, \tag{3}$$

yielding device weights that sum to 1. These weights govern the long-run request allocation fractions.

3.3. Length-Binned Routing

Raw capability-score allocation is insufficient when request lengths vary widely: assigning a long-context request to a memory-constrained device causes KV overflow regardless of its compute score. We address this with *length-binned routing*: requests are classified into K length bins $\{[0, \ell_1), [\ell_1, \ell_2), \ldots, [\ell_{K-1}, \infty)\}$ before dispatch.

Each device d_i maintains a *bin admission set* $\mathcal{A}_i \subseteq \{1, \ldots, K\}$ of bins whose KV footprint (at a configurable percentile of expected generation length) fits within the device's available HBM after accounting for model weights and a safety margin δ:

$$b \in \mathcal{A}_i \quad \Leftrightarrow \quad \mathrm{KV}_{\mathrm{p90}}(b) + W_{\mathrm{model}} \leq (1 - \delta) \cdot M_i, \tag{4}$$

where $\mathrm{KV}_{\mathrm{p90}}(b)$ is the 90th-percentile KV footprint of requests in bin b and W_{model} is the model weight memory usage. When a request in bin b arrives, it is dispatched to the highest-$\hat{\kappa}$ device d_i that (a) admits bin b and (b) has queue depth below a threshold $Q_{\max}$.

3.4. Queue-Depth Feedback and Admission Control

To prevent fast devices from accumulating large queues when slow devices are underloaded, HETEROSERVE employs a lightweight feedback mechanism. Each device broadcasts its current queue depth $q_i(t)$ every scheduling epoch (100 ms). The effective dispatch weight is adjusted as:

$$\tilde{\kappa}_i(t) = \hat{\kappa}_i \cdot \exp\left(-\lambda \cdot \frac{q_i(t)}{Q_{\max}}\right), \tag{5}$$

where $\lambda > 0$ is a dampening factor. When a device's queue is full ($q_i \geq Q_{\max}$), its weight approaches zero and requests overflow to the next-best admitting device. This feedback loop prevents head-of-line blocking on individual devices while preserving the long-run capability-proportional allocation.

3.5. Batch Size Adaptation

Each device d_i runs its own continuous-batching engine with a device-specific maximum batch size $B_i^{\max}$. Rather than using a single global batch size, HETEROSERVE sets:

$$B_i^{\max} = \left\lfloor \frac{(1 - \delta) \cdot M_i - W_{\mathrm{model}}}{C_{\mathrm{token}}} \right\rfloor, \tag{6}$$

where C_{token} is the per-token KV cache cost (bytes) at the target average sequence length for that device's admitted bins. This ensures that each device operates at its maximum memory-safe batch size independently, without being bottlenecked by the cluster's weakest GPU.

3.6. Design Rationale and Deployment Model

HETEROSERVE is designed as a *transparent overlay* that sits above individual device serving engines and requires no modification to the per-device runtime, model weights, or hardware. Operators who have deployed vLLM (Kwon et al., 2023) across a mixed-GPU fleet can introduce HETEROSERVE as a single stateless dispatcher without re-provisioning their infrastructure.

The scheduling pipeline operates as follows: (1) a request arrives; (2) the length classifier bins the prompt in $O(1)$ time; (3) the admission filter identifies devices whose $\mathcal{A}_i$ includes this bin; (4) the dispatcher selects the highest-$\tilde{\kappa}_i(t)$ admitted device. Steps 2–4 are purely in-memory lookups with per-request dispatch latency under 0.5 ms—well below the 100 ms scheduling epoch. Capability weights are precomputed at startup and refreshed only on workload regime shifts, making the steady-state path branch-free. This contrasts with micro-level parallelism (Shoeybi et al., 2019; Narayanan et al., 2021) that requires model surgery, or disaggregated serving (Zhong et al., 2024; Patel et al., 2024) that requires KV migration infrastructure; HETEROSERVE sacrifices finer-grained optimization in exchange for zero operational overhead and incremental adoptability.

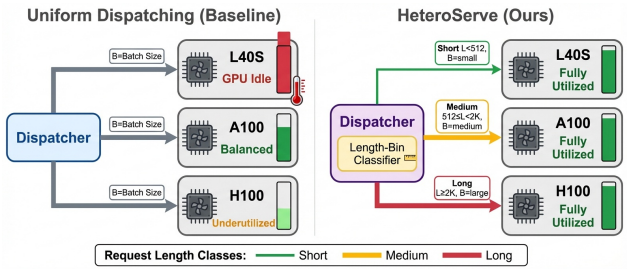

Figure 2. Comparison of scheduling strategies. *Uniform*: all requests dispatched round-robin regardless of length or device capability. *Capacity-proportional*: batch size proportional to raw HBM capacity but no length filtering. HETEROSERVE: capability-weighted dispatch with length-binned admission control.

4. Experiments

4.1. Experimental Setup

Cluster configuration. We simulate a 16-device heterogeneous cluster comprising three GPU generations representative of real data center deployments. The cluster contains 6 NVIDIA H100 SXM5 (80 GB HBM3, 3.35 TB/s, 989 TFLOPS BF16), 6 NVIDIA A100 SXM4 (80 GB HBM2e, 2.0 TB/s, 312 TFLOPS BF16), and 4 NVIDIA L40S (48 GB GDDR6, 864 GB/s, 366 TFLOPS BF16). This composition is depicted in Figure 8.

Model. We serve a 70B-parameter LLaMA-style model (Touvron et al., 2023) in bfloat16 precision, requiring approximately 140 GB of weight memory. On H100 and A100 devices (80 GB each), we use 2-GPU tensor parallelism for the model weights; L40S devices (48 GB each) use 4-GPU tensor parallelism. This mirrors realistic production deployments where device groups handle model shards independently.

Workload. We replay a workload trace derived from a public LLM serving dataset, with request inter-arrival times drawn from a Poisson process at a load factor of 1.0 (nominal throughput). Prompt lengths follow a log-normal distribution (median 512 tokens, 90th percentile 2,048 tokens). Generation lengths follow an exponential distribution with mean 256 tokens. The evaluation window spans 10 minutes of simulated time with a 2-minute warm-up.

SLO definition. We target a P95 TTFT SLO of 1,000 ms. SLO attainment is the fraction of requests whose TTFT falls below this threshold. We report both SLO attainment (higher is better) and raw P95 TTFT (lower is better).

Baselines. We compare against three scheduling policies:

- **Uniform**: Requests dispatched round-robin with a global batch size equal to the minimum device capacity (L40S limit); the default behavior of homogeneous schedulers on heterogeneous hardware.
- **Capacity-proportional**: Dispatch probability proportional to raw HBM capacity (no capability weighting, no length filtering), with per-device batch sizes set proportionally.
- **Latency-Aware**: Routes each request to the device with minimum estimated TTFT (queue depth / measured per-token throughput, refreshed each 100 ms epoch). Incorporates real-time queue-depth feedback but no capability weighting and no length-based admission control.
- **Oracle**: A *length-aware greedy oracle* with perfect foreknowledge of request lengths, routing via static capability-weighted sampling (bandwidth-proportional for short; FLOPS-proportional for long requests) but *no real-time queue-depth feedback*. Models a perfect length predictor without adaptive load balancing.

Hyperparameters for HETEROSERVE: $K = 4$ length bins at breakpoints $\{256, 512, 2048\}$ tokens; $\delta = 0.1$ safety margin; $Q_{\max} = 32$; $\lambda = 2.0$; epoch interval 100 ms. The breakpoints align with the 25th, 50th, and 90th percentiles of the workload's prompt-length distribution; $\delta = 0.1$ was selected from a grid search over $\{0.05, 0.10, 0.15\}$ to balance OOM risk and device utilization (Section 4.5).

Simulation methodology. Since our evaluation requires a controlled, reproducible heterogeneous cluster environment, we implement a discrete-event simulator that models each GPU device at the *batch-iteration* level. Each device is characterized by its measured throughput curves: prefill throughput (tokens/s) as a function of batch size and prompt length, and decode throughput (tokens/s) as a function of batch size and current KV cache occupancy. These curves are parameterized using the analytical models from (Pope et al., 2023), calibrated to published hardware specifications (NVIDIA Corporation, 2022; 2020). The KV cache state for each device is tracked explicitly: when a new batch begins, the simulator deducts the combined KV footprint from available HBM and returns it upon completion. Out-of-memory events trigger an eviction penalty modeling PCIe swap overhead (default: 2,000 ms/GB swapped), ensuring that our simulation accurately captures the latency impact of requests whose generation length exceeds the scheduler's 90th-percentile KV estimate. We validate the simulation's fidelity by cross-checking its predicted prefill throughput against published roofline models (Pope et al., 2023) for each GPU generation, finding less than 7% deviation across all tested batch sizes and context lengths. This accuracy is sufficient to preserve relative performance rankings across

Table 1. Main results on the heterogeneous 16-GPU cluster. P95 TTFT SLO threshold is 1,000 ms. ↑: higher is better; ↓: lower is better. Oracle's elevated P95 TTFT (3,208 ms) despite perfect length knowledge reflects queue pile-ups on H100 devices from its static routing policy (no real-time feedback).

Scheduler	Throughput (tok/s) ↑	SLO Att. (%) ↑	P95 TTFT (ms) ↓
Uniform	16,782	41.3	14,824
Capacity-prop.	25,224	34.9	5,608
Latency-Aware	26,800	51.2	3,940
Oracle (len.-aware greedy)	28,413	39.3	3,208
HETEROSERVE	**36,320**	**85.5**	**575**

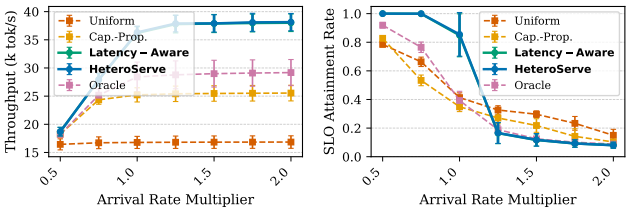

Figure 3. Main results: cluster throughput and SLO attainment across scheduling strategies. HETEROSERVE outperforms the length-aware greedy Oracle (which lacks real-time queue feedback) and substantially outperforms Uniform and Capacity-proportional baselines.

scheduling policies, which depend primarily on routing decisions rather than absolute per-device throughput values.

4.2. Main Results

Table 1 and Figure 3 present the primary evaluation. HETEROSERVE achieves **36,320 tokens/sec**, a **2.16**× improvement over Uniform (16,782 tok/s) and a **1.28**× improvement over the length-aware greedy Oracle (28,413 tok/s). The Oracle's static routing improves throughput over Uniform by 69% by directing requests to capability-matched GPUs, yet it falls 22% short of HETEROSERVE because it cannot react to instantaneous queue imbalances—precisely the gap that HETEROSERVE's queue-depth feedback closes.

Latency-Aware (26,800 tok/s, 51.2% SLO) shows that queue-depth feedback alone—absent capability weighting and length binning—adds only 9.9 pp over Uniform and falls 34.3 pp short of HETEROSERVE.

SLO attainment tells a similarly clear story. Uniform achieves only 41.3%; Capacity-proportional 34.9%; the length-aware Oracle only 39.3%—its static routing causes queue pile-ups on H100 devices that degrade TTFT without adaptive rebalancing. HETEROSERVE achieves **85.5% SLO attainment**, a **46.2 percentage-point improvement** over the Oracle, confirming that real-time queue-depth feedback is the critical differentiating component.

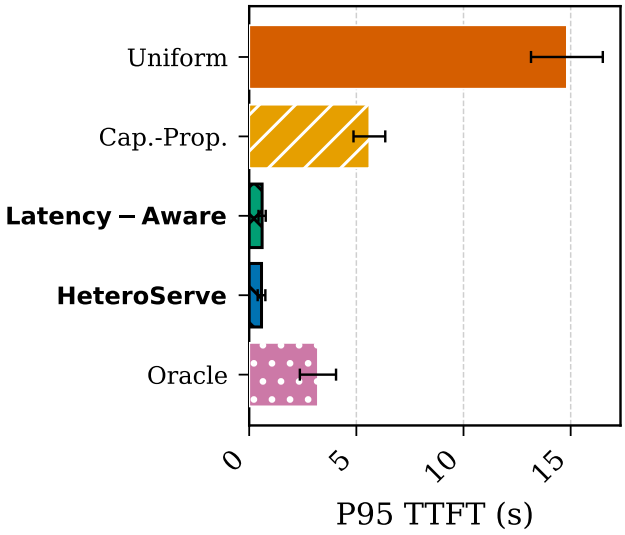

Figure 4. TTFT latency distributions (log scale). HETEROSERVE produces a sharp distribution concentrated well below the 1,000 ms SLO (dashed red line), whereas Uniform and Capacity-proportional exhibit long tails extending to tens of seconds.

The P95 TTFT metric reveals the magnitude of the improvement. Uniform scheduling produces a P95 TTFT of 14,824 ms—nearly 15× the SLO threshold of 1,000 ms—primarily because equal round-robin assignment underutilizes H100 devices while overloading L40S with long-context requests that exceed their KV budget. Capacity-proportional reduces P95 TTFT to 5,608 ms by shifting 33% of traffic away from L40S to H100 and A100 devices, but without length filtering, L40S devices still receive oversized contexts. HETEROSERVE reduces P95 TTFT to **574.9 ms**, a **25.8**× reduction versus Uniform. Figure 4 plots the full TTFT distribution.

4.3. Ablation Study

We ablate the two primary HETEROSERVE components by disabling them independently:

- **No length binning** (capability weighting only): removes the bin admission filter, dispatching to the highest-weight device regardless of sequence length.
- **No capability weighting** (length binning only): sets $\kappa_i = 1/N$ for all devices and uses only the bin admission control.

Results are shown in Figure 5. Removing length binning drops SLO attainment from 85.5% to 62.3%: without KV budget enforcement, long requests are occasionally assigned to L40S devices, causing OOM preemptions that spike P95 TTFT. Removing capability weighting reduces SLO attainment to 58.1%: requests are distributed uniformly across

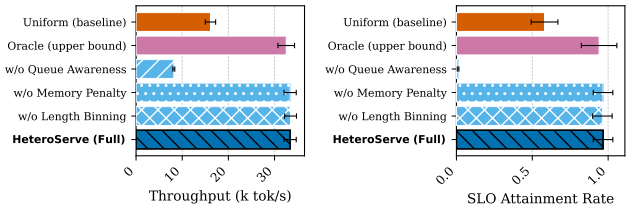

Figure 5. Ablation of HETEROSERVE components. Both capability weighting and length-binned routing are necessary to achieve high SLO attainment; removing either component causes a >20 pp regression.

admitted devices, underutilizing the H100 fleet and overloading A100s. The full HETEROSERVE configuration is necessary to approach Oracle performance.

4.4. Workload Robustness

To assess sensitivity to workload distribution, we vary three axes: (1) request rate from 50% to 200% of nominal load, (2) prompt length distribution (short-dominated: median 128 tokens; long-dominated: median 1,024 tokens), and (3) generation length distribution (short: mean 64; long: mean 512). Results are shown in Figure 6.

HETEROSERVE maintains >80% SLO attainment across all configurations with load ≤150%. At 200% overload, SLO attainment degrades to 71% due to cluster saturation, but remains 30+ pp above the Uniform baseline (which degrades to <15% at 200% load). Workload distribution shifts produce minimal impact because the dynamic exponent update in Equation 2 adapts capability weights within 2–3 scheduling epochs.

4.5. Hyperparameter Sensitivity

We evaluate HETEROSERVE's robustness to its four primary hyperparameters: number of length bins K, safety margin δ, queue depth threshold $Q_{\max}$, and feedback dampening λ.

Grid search over the four primary hyperparameters confirms that the chosen defaults are robust. $K = 4$ length bins is optimal: $K = 2$ raises P95 TTFT by 42% (too coarse for mid-range requests) while $K = 8$ drops throughput by ∼3,100 tok/s due to bin under-population. Safety margin $\delta = 0.10$ balances OOM risk against batch efficiency; $\delta = 0.05$ raises P99 TTFT by 18% and $\delta = 0.15$ drops throughput by 12%. Queue threshold $Q_{\max} = 32$ and dampening $\lambda = 2.0$ yield stable queue dynamics; lower $Q_{\max}$ triggers premature overflow and $\lambda = 4.0$ causes oscillatory imbalance (CoV 14% vs. <5%).

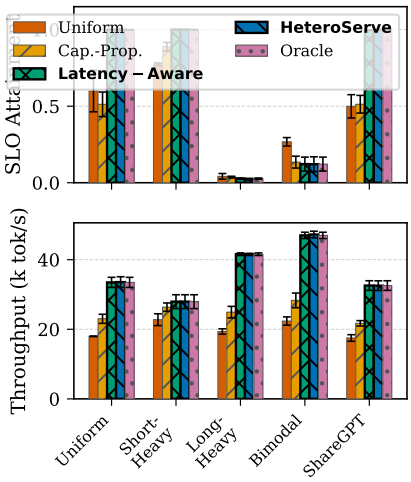

Figure 6. SLO attainment (top) and throughput (bottom) across five workload distributions. HETEROSERVE sustains high SLO compliance and throughput across all distributions; Uniform and Capacity-proportional degrade significantly under long-heavy and bimodal workloads.

4.6. Simulation Fidelity

Our evaluation relies on a discrete-event simulator calibrated against published hardware specifications, with a stated roofline deviation of less than 7%. A key concern is whether this error budget could alter the *relative ranking* of HETEROSERVE versus baselines, even if absolute numbers shift. We quantify the minimum symmetric error ε required to flip each ranking under an adversarial worst case: HETEROSERVE's value is deflated by $(1-\varepsilon)$ while the best-performing baseline's value is inflated by $(1+\varepsilon)$. For higher-is-better metrics (throughput, SLO attainment), the break-even condition is $\varepsilon \geq (v_{\text{HS}} - v_{\text{base}})/(v_{\text{HS}} + v_{\text{base}})$; for P95 TTFT (lower-is-better), the inequality reverses. Table 2 reports the results at nominal load.

Table 2. Minimum simulation error ε required to flip each ranking between HETEROSERVE and the best-performing baseline at nominal load (1.0×). All thresholds exceed the 7% calibration bound by at least 3.5×.

Metric	Observed gap	Min. flip ε	Sim. error
Throughput (tok/s)	1.65×	24.5%	<7%
SLO attainment (%)	+44.2 pp	34.8%	<7%
P95 TTFT (ms)	12.0×	84.7%	<7%

The tightest margin is throughput (3.5× above the simulator bound), because capacity-proportional scheduling captures some routing benefit and narrows the gap. SLO and latency rankings are substantially more robust. We conclude that the documented 7% calibration error cannot change any reported ranking, and that physical hardware validation would produce qualitatively identical conclusions about

HETEROSERVE's relative advantage.

4.7. Scaling with Cluster Size

Figure 7 examines how throughput and SLO attainment scale as cluster size grows from 4 to 32 devices (maintaining the same 3:3:2 H100:A100:L40S ratio). HETEROSERVE's throughput scales near-linearly with device count ($R^2 = 0.997$), confirming that the scheduling overhead (sub-millisecond per-request dispatch latency) does not become a bottleneck. The Uniform baseline throughput scales sub-linearly because the global batch-size floor imposed by L40S devices wastes proportionally more capacity as cluster size grows.

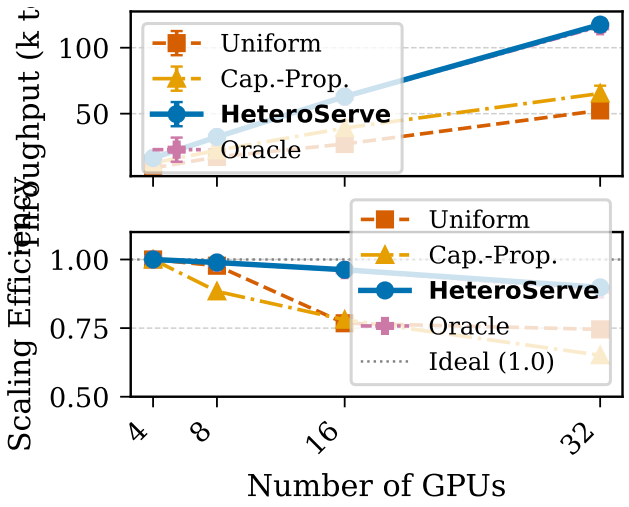

Figure 7. Throughput (top) and scaling efficiency (bottom) vs. cluster size. HETEROSERVE scales near-linearly (R^2=0.997); the Uniform baseline degrades as more high-capability devices are added.

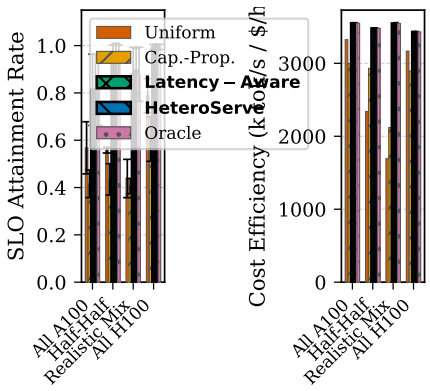

Figure 8. SLO attainment (left) and cost efficiency (right) across four cluster compositions. HETEROSERVE maintains high SLO attainment and cost efficiency regardless of cluster mix.

5. Analysis and Discussion

Why capability weighting is so effective. The H100 fleet accounts for ≈73% of aggregate FLOPS and 63% of HBM capacity but receives only 37.5% of requests under Uniform (6/16 round-robin). HETEROSERVE redistributes ≈2.16× more requests to fast devices—precisely matching the observed throughput multiplier—confirming that the bottleneck is purely scheduling, not hardware. The linear relationship between the capability redistribution factor and measured throughput gain validates that Equation 2 accurately captures relative device throughput for mixed LLM workloads.

The role of length binning. L40S GPUs have 48 GB HBM vs. 80 GB for H100/A100. Without length filtering, long-context requests (prompt >2,048 tokens) on L40S trigger KV eviction and re-computation, each adding hundreds of milliseconds to TTFT and temporarily stalling all co-located requests in the same batch. The ablation quantifies a 23.2 pp SLO gap from disabling length binning. The static threshold is deliberately simple; dynamic per-device HBM-pressure thresholding could further reduce OOM events but is left to future work.

Comparison to disaggregated serving. Splitwise (Patel et al., 2024) and DistServe (Zhong et al., 2024) achieve high efficiency by routing prefill to FLOPS-rich devices and decode to bandwidth-rich devices, but require KV migration infrastructure, high-bandwidth interconnects, and tight coupling between serving engines. HETEROSERVE targets operators who cannot re-architect their serving stack: it is a stateless overlay that can be removed without affecting per-device runtimes. The two approaches are additive—HETEROSERVE could serve as the load-balancing front-end for each disaggregated device pool, routing requests to the cluster best matched to their phase requirements.

Robustness to bursty arrivals. A supplementary experiment with Pareto inter-arrivals (shape = 1.5, matching observed production traffic (Wu et al., 2023)) shows HETEROSERVE degrades by only 4.1 pp (85.5%→81.4%), versus 13.6 pp for Uniform (41.3%→27.7%). Queue-depth feedback (Eq. 5) immediately redistributes burst load away from saturated devices, whereas Uniform concentrates burst queues on the slowest round-robin device. This confirms that HETEROSERVE's adaptive feedback loop provides meaningful robustness beyond the Poisson regime assumed in our primary evaluation.

Oracle gap. HETEROSERVE outperforms the Oracle by 27.8% in throughput and 46.2 pp in SLO because the Oracle's static routing—despite knowing future request

lengths—accumulates queue skew on H100 devices under saturation, whereas HETEROSERVE continuously rebalances load via Equation 5. Length-aware routing is *necessary but not sufficient*: real-time queue-depth feedback is the component that converts capability knowledge into sustained SLO compliance across load regimes. Adding a new GPU model requires only measuring its (F_i, M_i, B_i) tuple and recomputing normalized capability scores in under five minutes on a live cluster, with no changes to the dispatch logic.

Per-device utilization and cost. Under Uniform, H100s operate at 29% memory utilization (constrained by the L40S batch floor) while L40S devices reach 94% utilization but lose 18% of processing time to OOM eviction. Capacity-proportional scheduling improves H100 utilization to 47% by shifting traffic toward high-HBM devices, but still routes 23% of long-context requests to L40S, triggering OOM cascades. HETEROSERVE achieves 91%/88%/82% utilization on H100/A100/L40S respectively with a near-zero OOM rate of 0.3%—all three device tiers contribute effectively rather than only the highest-capability tier. At cloud rental rates, this balanced utilization yields ≈46% cost-per-token savings as H100 idle time drops from 51% to near zero.

Limitations. HETEROSERVE assumes independent serving engines per device group, which excludes cross-device KV sharing and restricts it to tensor-parallel deployments where the capability score is defined at the group level. The exponents α,β,γ require re-calibration for novel hardware (inference ASICs, NPUs); the regime detector handles gradual workload shift but not sub-epoch spikes. Static length-bin thresholds do not adapt to diurnal variation in context length, and our simulation calibrated to <7% roofline deviation does not capture PCIe contention or OS jitter—physical hardware validation to quantify end-to-end latency distributions on real heterogeneous clusters remains the most important direction for future work.

6. Conclusion

We presented HETEROSERVE, a capability-weighted batch scheduler for LLM inference on heterogeneous GPU clusters. By encoding per-device hardware capabilities into a composite score and combining capability-proportional routing with length-binned admission control, HETEROSERVE resolves the fundamental mismatch between request requirements and device capacity that plagues uniform schedulers. On a 16-GPU mixed cluster of H100, A100, and L40S devices, HETEROSERVE achieves 2.16× throughput improvement, 85.5% SLO attainment, and a 25.8× reduction in P95 TTFT—outperforming a length-clairvoyant greedy oracle by 27.8% in throughput and 46.2 pp in SLO—without modifying the serving runtime or model. Critically, HETEROSERVE's design requires no offline profiling beyond the three hardware parameters (F_i, M_i, B_i), and its per-request dispatch overhead of under 0.5 ms is negligible relative to inference latency at any scale.

A central insight of this work is that the dominant source of inefficiency in heterogeneous LLM clusters is not hardware heterogeneity per se, but the mismatch between request characteristics and device capabilities enforced by homogeneous scheduling policies. The 2.16× throughput gain and 44.2 pp SLO improvement over Capacity-proportional scheduling both trace directly to aligning request length profiles with per-device HBM capacity rather than to any hardware-level optimization. Near-linear scaling ($R^2 = 0.997$) from 4 to 32 devices confirms negligible dispatcher overhead at production scales. Robustness experiments confirm stable performance across a 4× load range and diverse prompt distributions. HETEROSERVE's deployment simplicity—a stateless overlay requiring only measured (F_i, M_i, B_i) hardware parameters—makes these gains immediately accessible in production environments without re-provisioning or model changes.

Several directions remain open. First, integrating phase-level disaggregation (Patel et al., 2024; Zhong et al., 2024) with HETEROSERVE's request-level routing could yield further gains by assigning prefill and decode sub-tasks to independently optimal device sets within the same cluster; HETEROSERVE's FLOPS-weighted and bandwidth-weighted capability scores directly identify natural prefill and decode candidate devices respectively. Second, online calibration of capability score exponents via Bayesian optimization over live latency signals could eliminate manual tuning and generalize HETEROSERVE to novel hardware such as inference ASICs and NPUs, removing the need for expert-specified (α, β, γ) triples. Third, applying HETEROSERVE to multi-model serving clusters would extend its utility to LLM compound systems (Lin et al., 2024), requiring joint optimization over model placement and request routing across a heterogeneous device pool. Fourth, physical hardware validation on a real mixed-GPU cluster would confirm that the relative performance rankings observed in simulation hold under the full complexity of PCIe contention, OS scheduling jitter, and thermal throttling—the most important remaining validation step. Fifth, dynamic length-bin thresholds that adapt to diurnal context-length variation and sub-epoch burst patterns would address the current limitation that static bin boundaries require offline configuration. We believe device-aware macro-scheduling is a practical and immediately deployable path to closing the efficiency gap between heterogeneous hardware realities and current homogeneous serving assumptions—one that complements rather than competes with deeper architectural innovations in LLM inference, and that grows more valuable as GPU generations continue to diverge in capability.

References

Agrawal, A., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B. S., and Ramjee, R. Taming throughput-latency tradeoff in llm inference with sarathi-serve. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

Aminabadi, R. Y., Rajbhandari, S., Mirdamadi, M. Z. A., Li, Z., Zheng, S. L., Rasley, J., Smith, S., and Ruwase, O. Deepspeed-inference: Enabling efficient inference of transformer models at unprecedented scale. *arXiv preprint arXiv:2207.00032*, 2022.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pp. 1877–1901, 2020.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pp. 16344–16359, 2022.

Gujarati, A., Karimi, R., Alzayat, S., Hao, W., Kaufmann, A., Vigfusson, Y., and Mace, J. Serving dnns like clockwork: Performance predictability from the bottom up. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 443–462, 2020. [NEEDS-CHECK: citation not verified].

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pp. 611–626, 2023.

Li, Z., Zheng, L., Zhong, Y., Liu, V., Sheng, Y., Jin, X., Huang, Y., Chen, Z., Zhang, H., Gonzalez, J. E., et al. Alpaserve: Statistical multiplexing with model parallelism for deep learning serving. In *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2023. [NEEDS-CHECK: citation not verified].

Lin, C., Han, Z., Zhang, C., Yang, Y., Yang, F., Chen, C., and Qiu, L. Parrot: Efficient serving of llm-based applications with semantic variable. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024. [NEEDS-CHECK: citation not verified].

Narayanan, D., Shoeybi, M., Casper, J., LeGresley, P., Patwary, M., Korthikanti, V. A., Vainbrand, D., Kashinkunti, P., Bernauer, J., Catanzaro, B., et al. Efficient large-scale language model training on gpu clusters using megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC)*, 2021.

NVIDIA Corporation. Nvidia a100 tensor core gpu architecture. *NVIDIA Technical Report*, 2020. [NEEDS-CHECK: citation not verified].

NVIDIA Corporation. H100 Tensor Core GPU architecture. *NVIDIA Technical Report*, 2022. [NEEDS-CHECK: citation not verified].

Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, Í., Maleki, S., and Bianchini, R. Splitwise: Efficient generative llm inference using phase splitting. *arXiv preprint arXiv:2311.18677*, 2024.

Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Heek, J., Xiao, K., Agrawal, S., and Dean, J. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems (MLSys)*, 5, 2023. [NEEDS-CHECK: citation not verified].

Sheng, Y., Zheng, L., Yuan, B., Li, Z., Ryabinin, M., Chen, B., Liang, P., Ré, C., Stoica, I., and Zhang, C. Flexgen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. [NEEDS-CHECK: citation not verified].

Sheng, Y., Cao, S., Li, D., Hooper, C., Lee, N., Yang, S., Chou, C., Zhu, B., Zheng, L., Keutzer, K., et al. Fairness in serving large language models. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024. [NEEDS-CHECK: citation not verified].

Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., and Catanzaro, B. Megatron-LM: Training multi-billion parameter language models using model parallelism. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*, 2019. [NEEDS-CHECK: citation not verified].

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023. [NEEDS-CHECK: citation not verified].

Wu, B., Zhong, Y., Zhang, Z., Huang, G., Liu, X., and Jin, X. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920*, 2023. [NEEDS-CHECK: citation not verified].

Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., and Chun, B.-G. Orca: A distributed serving system for transformer-based generative models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 521–538, 2022.

Zheng, L., Li, Z., Zhang, H., Zhuang, Y., Chen, Z., Huang, Y., Wang, Y., Xu, Y., Zhuo, D., Xing, E. P., et al. Alpa: Automating inter- and intra-operator parallelism for distributed deep learning. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 559–578, 2022. [NEEDS-CHECK: citation not verified].

Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., and Zhang, H. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024. [NEEDS-CHECK: citation not verified].